

LYCÉE CÉSAR BAGGIO

TERM STI2D SIN

TRAVAUX PRATIQUES

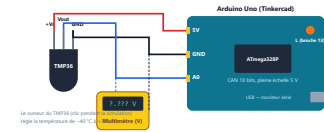
# Surveiller la température d'une salle serveur — capteur TMP36 et CAN de l'Arduino

*Une alerte avant que les serveurs ne chauffent.*

**Durée** 2 h 00 min

**Compétences** CO7.3-SIN1 · CO5.8-SIN2 · CO7.2 · 2.4.2a · 2.4.2c · 5.3.1d

- TP noté, en binôme. Appeler le professeur aux points de validation 🍷.
- Rédiger les calculs : formule, application numérique, résultat avec unité.



Nom .....

Prénom .....

Classe .....

Date .....

/ 20

## 1. Contextualisation & Système Étudié

### LA SALLE SERVEUR DU LYCÉE

Les serveurs du lycée (ENT, réseau pédagogique) sont regroupés dans une petite salle climatisée. Si la climatisation tombe en panne, la température monte vite et les serveurs risquent de s'arrêter. Le service informatique veut une **alerte dès que la température dépasse 27 °C**, et un **affichage de la température à 0,5 °C près**.

On prototypé la solution dans **Tinkercad Circuits** avec une carte Arduino Uno et un capteur de température analogique TMP36.

### PROBLÉMATIQUE

La chaîne capteur TMP36 + CAN de l'Arduino permet-elle de mesurer la température à 0,5 °C près ? Comment déclencher l'alerte ?

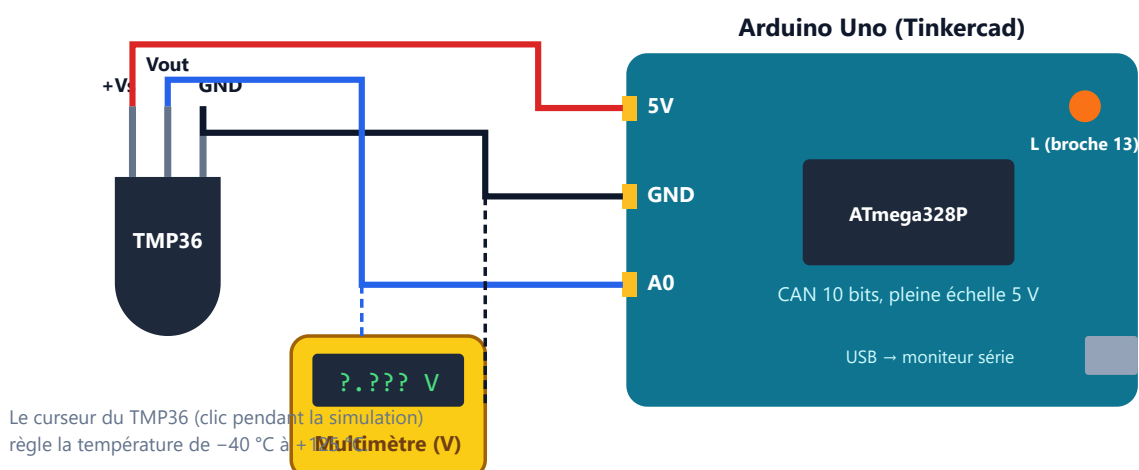


Figure 1 — Montage à réaliser dans Tinkercad

**DR1** Capteur de température TMP36 (extrait simplifié)

| Caractéristique   | Valeur                                                                                     |
|-------------------|--------------------------------------------------------------------------------------------|
| Alimentation      | 2,7 V à 5,5 V                                                                              |
| Étendue de mesure | -40 °C à +125 °C                                                                           |
| Tension de sortie | $V_{out} = 0,5 \text{ V} + 0,01 \text{ V}/^{\circ}\text{C} \times T$ (soit 750 mV à 25 °C) |
| Précision typique | $\pm 2 \text{ }^{\circ}\text{C}$                                                           |

**DR2** CAN de la carte Arduino Uno

| Caractéristique                       | Valeur                                                         |
|---------------------------------------|----------------------------------------------------------------|
| Nombre de bits                        | 10                                                             |
| Pleine échelle (référence par défaut) | 5 V                                                            |
| Fonction de lecture                   | <code>analogRead(A0)</code> renvoie le code $N$ , de 0 à 1 023 |

## 2. Travail Demandé

### Partie A — Prévoir ( $\approx 25$ min)

**Question 1**

2 pts

À l'aide du DR1, calculer la tension de sortie du TMP36 pour 20 °C, 27 °C, -40 °C et 125 °C.

**Question 2**

2 pts

Calculer le quantum  $q$  du CAN de l'Arduino (DR2), puis la plus petite variation de température que la chaîne peut détecter. Le cahier des charges (affichage à 0,5 °C près) est-il respecté ?

**Question 3**

1 pt

Quel code  $N$  le CAN doit-il fournir au seuil d'alerte de 27 °C ?

## Partie B — Réaliser et mesurer (≈ 1 h)

### Question 4

2 pts

Réaliser le montage dans Tinkercad : TMP36 alimenté en 5 V, sortie  $V_{out}$  sur l'entrée **A0**, multimètre en voltmètre entre  $V_{out}$  et GND. Lancer la simulation, régler le capteur sur **25 °C** et relever la tension affichée par le multimètre. Comparer à la valeur attendue.

👉 **Validation professeur** : montage et mesure.

### Question 5

4 pts

Copier le programme de départ dans l'éditeur de code (mode « Texte »), puis compléter les zones **TODO Q5** pour calculer la tension **v** puis la température **t** à partir du code **n**. Afficher le résultat dans le moniteur série.

```
// station_temperature.ino - PROGRAMME DE DEPART
const int PIN_CAPTEUR = A0;
const int PIN_ALARME = 13; // LED "L" integree a la carte
const float PLEINE_ECHELLE = 5.0; // pleine echelle du CAN (V)
const int NB_CODES = 1024; // CAN 10 bits
const float SEUIL = 27.0; // seuil d'alerte (degC)

void setup() {
  Serial.begin(9600);
  pinMode(PIN_ALARME, OUTPUT);
}

void loop() {
  int n = analogRead(PIN_CAPTEUR); // code numerique de 0 a 1023

  // TODO Q5 : calculer la tension v (en V) a partir de n
  float v = 0;

  // TODO Q5 : calculer la temperature t (en degC) a partir de v
  float t = 0;

  Serial.print("N = ");
  Serial.print(n);
  Serial.print(" V = ");
  Serial.print(v, 3);
  Serial.print(" V T = ");
  Serial.print(t, 1);
  Serial.println(" degC");

  // TODO Q7 : allumer la LED d'alarme si t depasse SEUIL, l'eteindre sinon

  delay(1000);
}
```

**Question 6****4 pts**

Régler successivement le capteur aux températures du tableau. Relever le code  $N$  et la température calculée par le programme, puis calculer l'écart. Conclure : l'écart est-il cohérent avec la résolution calculée en Q2 ?

| Température réglée (°C)   | -10 | 0 | 25 | 27 | 50 |
|---------------------------|-----|---|----|----|----|
| Code $N$ lu               |     |   |    |    |    |
| Température calculée (°C) |     |   |    |    |    |
| Écart (°C)                |     |   |    |    |    |

👉 **Validation professeur** : tableau rempli.

**Question 7****2 pts**

Compléter la zone `TODO Q7` pour que la LED « L » s'allume au-dessus de 27 °C. Tester en faisant varier la température autour du seuil.

👉 **Validation professeur** : alerte fonctionnelle.

**Partie C — Améliorer (≈ 20 min)****Question 8****3 pts**

Le service informatique veut maintenant un affichage à **0,1 °C près**.

- En gardant une pleine échelle de 5 V, combien de bits faudrait-il au CAN au minimum ?
- Proposer une autre solution qui ne change pas le nombre de bits.

### 3. Bilan & Synthèse des Acquis

#### À RETENIR

- Le CAN de l'Arduino est un CAN \_\_\_\_\_ bits de pleine échelle 5 V : son quantum vaut environ \_\_\_\_\_.
- La résolution de la chaîne en température vaut  $\frac{q}{s}$  : environ \_\_\_\_\_ avec le TMP36.
- Pour améliorer la résolution : plus de \_\_\_\_\_, une pleine échelle plus \_\_\_\_\_, ou une amplification du signal.

### 4. Barème & Grille d'Évaluation

| Compétence                                                | Questions   | Critères                                                                                                   | Points    |
|-----------------------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------|-----------|
| <b>CO7.3-SIN1</b> — Expérimenter une chaîne d'acquisition | Q1 à Q4, Q6 | Calculs justes et justifiés, montage fonctionnel, mesures relevées, écarts expliqués par la quantification | 11        |
| <b>CO5.8-SIN2</b> — Programmer la réponse logicielle      | Q5, Q7      | Conversion code → tension → température juste (pas de division entière), alerte fonctionnelle              | 6         |
| <b>CO7.2</b> — Valider et proposer une amélioration       | Q8          | Nombre de bits justifié, solution alternative pertinente                                                   | 3         |
| <b>Total</b>                                              |             |                                                                                                            | <b>20</b> |